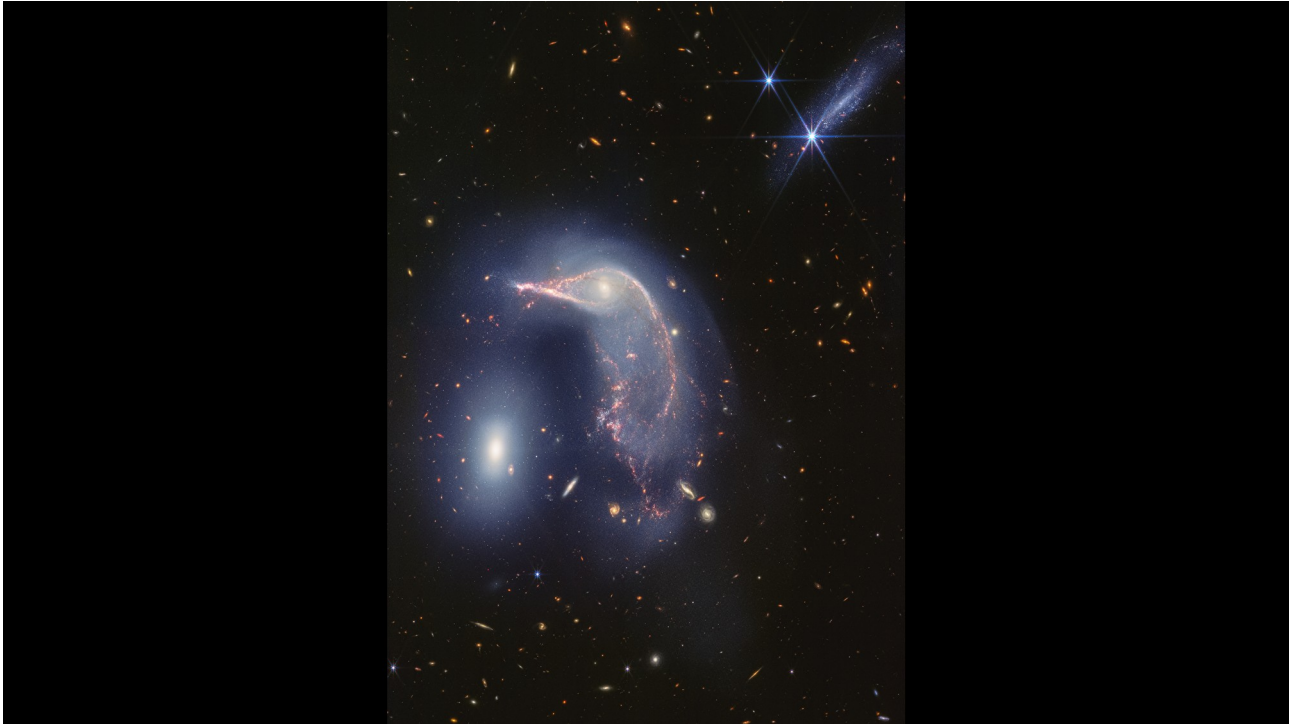


Spaceshield Hack 2026. Zadanie Specjalne od Aldec.

W równoległej czasoprzestrzeni naukowcy z planety Sol III postanowili raz na zawsze odpowiedzieć na pytanie, które od pokoleń dzieliło mieszkańców tej błękitnej planety:

Co było pierwsze - jajko czy... pingwin?



Para oddziałujących na siebie galaktyk (Arp 142)

źródło: <https://science.nasa.gov/asset/webb/interacting-galaxies-arp-142-hubble-and-webb-image/>

Aby rozwikłać tę zagadkę, zorganizowano międzygwiazdną ekspedycję badawczą. Jej celem był układ galaktyk Arp 142, gdzie dwa niezwykle obiekty oddziaływały na siebie w sposób budzący zainteresowanie całej społeczności naukowej. Były to galaktyki NGC 2936, przypominająca kształtem Pingwina, oraz NGC 2937, znana jako Jajo. Ich powolne zbliżanie się mogło skrywać odpowiedź na odwieczne pytanie.

Na pokładzie statku kosmicznego Artemis III umieszczono najnowocześniejsze instrumenty badawcze, a rakieta nośna bezpiecznie wyniosła misję poza atmosferę Sol III. Po wielu miesiącach podróży Artemis III dotarł do celu i rozpoczął przygotowania do transmisji danych na stację badawczą na Ziemi.

Wtedy wydarzyło się coś nieprzewidzianego.

Uszkodzeniu uległ moduł nadawczy statku. Łączność została zerwana, a dodatkowo awaria spowodowała zmianę częstotliwości pracy odbiornika. Dane, które mogły przynieść odpowiedź całej cywilizacji, utknęły w odległej przestrzeni kosmicznej.

W odpowiedzi na kryzys zorganizowano specjalne wydarzenie, podczas którego najlepsze zespoły inżynierów i programistów miały podjąć próbę odzyskania komunikacji z Artemis III.

Waszym zadaniem jest stworzenie algorytmu, który umożliwi ponowne nawiązanie łączności pomiędzy stacją badawczą na Sol III a statkiem kosmicznym Artemis III.

Czas ucieka. Sygnał słabnie. A odpowiedź wciąż czeka.

Teoria:

1. Rozpoznanie magistrali AXI4-Lite

AXI4-Lite jest standardem wykorzystywanym w rzeczywistych systemach FPGA do konfiguracji rejestrów sterujących. W praktyce właśnie w ten sposób procesory zarządzają peryferiami. W tej misji AXI4-Lite pełni rolę kanału dowodzenia.

Co należy zrozumieć:

AXI4-Lite składa się z pięciu niezależnych kanałów:

Zapis adresu	Zapis danych	Odpowiedź zapisu	Odczyt adresu	Odczyt danych
AWADDR	WDATA	BRESP	ARADDR	RDATA
AWVALID	WSTRB	BVALID	ARVALID	RRESP
AWREADY	WVALID	BREADY	ARREADY	RVALID
-	WREADY	-	-	RREADY

Tabela sygnałów interfejsu AXI4-Lite

Każda operacja na sygnałach AXI write/read odbywa się w niezależnych kanałach, jednak musi być realizowana ze świadomością zależności czasowych oraz wymaganej kolejności transakcji. Aby interfejs działał poprawnie, należy zachować właściwą sekwencję handshake pomiędzy kanałami adresowymi, danymi i odpowiedzi, z uwzględnieniem poprawnej synchronizacji sygnałów VALID oraz READY.

Dodatkowe informacje na temat sygnałów protokołu AXI4-Lite są dostępne w dokumentacji dołączonej do projektu.

2. Sterowanie przez Aldec BFM

Czym jest BFM?

Bus Functional Model (BFM) to symulacyjny model sterownika magistrali, umożliwiający generowanie transakcji bez konieczności ręcznej obsługi każdego sygnału interfejsu z osobna. Zamiast sterowania liniami sygnałowymi można korzystać z operacji wysokiego poziomu, takich jak:

- Write(adres, dane)
- Read(adres)

Takie podejście znacząco upraszcza tworzenie testbenchy, ogranicza liczbę zbędnego kodu oraz ułatwia rozwój większych projektów. Dodatkową zaletą jest szybszy debug, prostsze zarządzanie scenariuszami testowymi oraz lepsza czytelność środowiska weryfikacyjnego.

BFM-y odwzorowują rzeczywiste transakcje magistralowe, dzięki czemu umożliwiają wiarygodną symulację komunikacji z projektowanym układem.

W środowisku Aldec dostępne jest wsparcie dla wielu wariantów interfejsu AXI. W tym przykładzie wykorzystywany będzie model AXI4-Lite BFM. Poniżej przedstawiono przykładowe funkcje:

BfmWriteAddress - generuje transakcję AXI z podanymi danymi wejściowymi na kanale zapisu adresu. Przyjmuje dwa parametry: awaddr oraz awprot. Wartość parametru awprot domyślnie powinna być ustawiona na 0.

BfmReadAddress - generuje transakcję AXI z podanymi danymi wejściowymi na kanale odczytu adresu. Przyjmuje dwa parametry: araddr oraz arprot. Wartość parametru arprot domyślnie powinna być ustawiona na 0.

BfmWriteData - generuje pojedynczą transakcję AXI z podanymi danymi wejściowymi na kanale zapisu danych. Przyjmuje dwa parametry: wdata i wstrb. Wartość parametru wstrb powinna być domyślnie ustawiona na wartość maksymalną.

BfmWaitForWriteResponse - steruje sygnałem BREADY i oczekuje na odpowiedź zapisu. Przyjmuje parametr bresp, który domyślnie powinien być ustawiony na 0.

BfmWaitForReadResponse - steruje sygnałem RREADY i oczekuje na odpowiedź odczytu i dane. Przyjmuje parametr rresp, który domyślnie powinien być ustawiony na 0.

Poszczególne funkcje zostały opisane w dokumencie o nazwie ALDEC_BFM_AMBA_AXI.pdf, który został dołączony do projektu.

3. Dynamiczna rekonfiguracja w czasie pracy układu

W warunkach misji kosmicznej nie ma możliwości fizycznej ingerencji w działający system po wyniesieniu go na orbitę. Gdy statek pozostaje aktywny, a warunki pracy zmieniają się w trakcie lotu, konieczne może być bieżące dostosowywanie parametrów elektroniki pokładowej bez zatrzymywania urządzenia. W takich zastosowaniach szczególnie istotne stają się mechanizmy umożliwiające dynamiczne strojenie układów.

Dynamic Reconfiguration Port (DRP) to interfejs konfiguracyjny umożliwiający modyfikację wybranych rejestrów wewnętrznych układu podczas pracy systemu, bez konieczności ponownej konfiguracji całego FPGA. Dzięki temu możliwa jest dynamiczna zmiana parametrów bloków sprzętowych.

Komunikacja przez DRP polega na wykonaniu operacji zapisu lub odczytu wskazanego adresu rejestru. Typowa sekwencja obejmuje ustawienie adresu, określenie rodzaju operacji, podanie danych (dla zapisu), uruchomienie transakcji oraz oczekiwanie na sygnał potwierdzający jej zakończenie. Zachowanie właściwej kolejności sygnałów sterujących jest wymagane do poprawnej pracy interfejsu.

Jednym z bloków sprzętowych, które należy skonfigurować aby dostroić częstotliwość jest Mixed-Mode Clock Manager (MMCM). MMCM to blok zarządzania zegarem dostępny w układach FPGA. Jednymi z głównych funkcji MMCM jest mnożenie i dzielenie częstotliwości. MMCM korzysta z liczników mnożących/dzielących, które wyznaczają częstotliwość wyjściową układu.

Blok MMCM ma restrykcyjne warunki pracy, jego częstotliwość wewnętrzna (F_{VCO}) nie może być zbyt niska ani zbyt wysoka. Zakres częstotliwości wewnętrznej wynosi od 600MHz do 1200MHz. Częstotliwość wewnętrzna jest uzyskiwana poprzez wykonanie operacji mnożenia i dzielenia częstotliwości wejściowej. F_{VCO} można obliczyć z poniższego wzoru:

$$F_{VCO} = F_{WE} \cdot \frac{M}{D}$$

Gdzie:

F_{VCO} - częstotliwość wewnętrzna MMCM [MHz]

F_{WE} - częstotliwość zegara wejściowego [MHz]

M - mnożnik

D - dzielnik wejściowy

Do obliczeń należy przyjąć F_{WE} równą 125MHz.

Mnożnik może przyjmować wartości z zakresu od 2 do 64, z krokiem równym 0,125. Wejściowy dzielnik może natomiast przyjmować wyłącznie wartości całkowite z przedziału od 1 do 106.

Aby uzyskać wymaganą częstotliwość wyjściową F_{OUT} , czyli tę wykorzystywaną w odbiorniku Artemis III, należy odpowiednio podzielić częstotliwość wewnętrzną za pomocą dodatkowego dzielnika wyjściowego (clkout). Parametr tego dzielnika może przyjmować wartości od 1 do 128 i, podobnie jak mnożnik, może być liczbą ułamkową z rozdzielczością 0,125.

$$F_{OUT} = \frac{F_{VCO}}{O}$$

O – dzielnik kanału clkout

Instrukcja do wykonania zadania:

1. Obliczenie częstotliwości nośnej [10 pkt]

Częstotliwość nośna została zaszyfrowana przez pokładowy komputer Artemis III z użyciem fundamentalnych parametrów układu Sol III. Jedynie poprawne odtworzenie wzoru umożliwi ponowne zsynchronizowanie kanału transmisyjnego.

$$f_{out} = \frac{100 \cdot \sqrt{\frac{T_{SID}^2}{\left(\frac{86164}{4}\right)^2} + \frac{\cos^2 \theta + \sin^2 \theta}{4} \cdot \frac{D_{MOON}}{\sqrt{31^4}}} + \sqrt{\frac{D_{AU}}{93498,6691875}} + 13 \cdot \sqrt{\frac{Y_{EARTH}}{1461,4}} + \frac{\log_2 \sqrt[8]{2^5}}{10}}{V_{FPGA}}$$

Gdzie:

f_{out} - częstotliwość wyjściowa nadajnika [MHz]

T_{SID} - czas trwania doby gwiazdowej Ziemi z dokładnością do sekundy [s]

θ - kąt nachylenia osi ziemskiej do ekliptyki [°]

D_{MOON} - średnia odległość Ziemi od Księżyca [km]

D_{AU} - dokładnie 1 jednostka astronomiczna [km]

Y_{EARTH} - czas trwania roku ziemskiego syderycznego z dokładnością do części setnych [dni]

V_{FPGA} - wartość napięcia układu FPGA odbiornika [V]

Najnowsze czasopismo poświęcone galaktykom opublikowało informację, że częstotliwość wyjściowa wyrażona w MHz w wielu przypadkach może przyjmować wartość zbliżoną do połowy odległości do danej galaktyki (mierzonej w latach świetlnych). Niestety, Łowcy Mitów nie zweryfikowali tego intrygującego faktu.

Aby określić wartość napięcia układu FPGA odbiornika, należy odczytać zawartość rejestru FPGA_VOLTAGE. W tym celu można skorzystać z funkcji BFM od Aldec. Szczegółowe informacje dotyczące tego rejestru znajdują się w dokumentacji AXI_CLK_Generator_HDL.pdf.

2. Uruchomienie symulatora Riviera-PRO [5 pkt]

Należy uruchomić symulator Riviera-PRO i ustawić folder: Spaceshield_Hack_2026_Aldec/project/riviera jako folder roboczy.

Aby rozpocząć pracę z symulatorem Riviera-PRO, konieczna jest jego instalacja oraz skonfigurowanie aktywnej licencji.

Pobieranie i Instalacja:

Pobierz plik instalacyjny z serwera dostępnego pod poniższym adresem:

Windows: Wybierz plik z rozszerzeniem .exe.

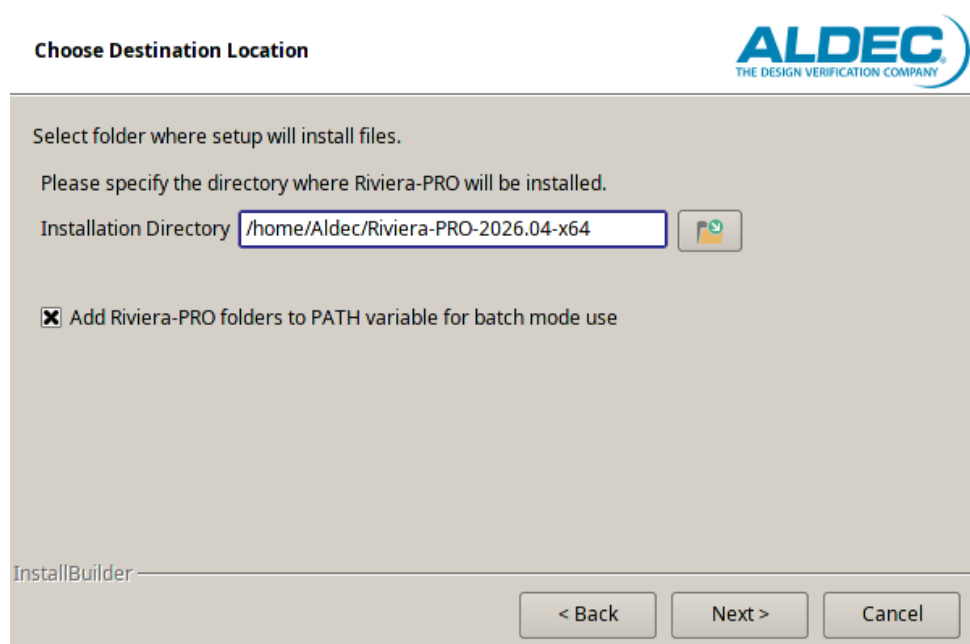
Linux: Wybierz plik z rozszerzeniem .run.

Hasło do obu plików to:

StalowaWola#2026

Podczas instalacji zaleca się pozostawienie domyślnych ustawień. Należy jednak koniecznie upewnić się, że zaznaczona jest opcja:

Add Riviera-PRO folders to PATH variable for batch mode use



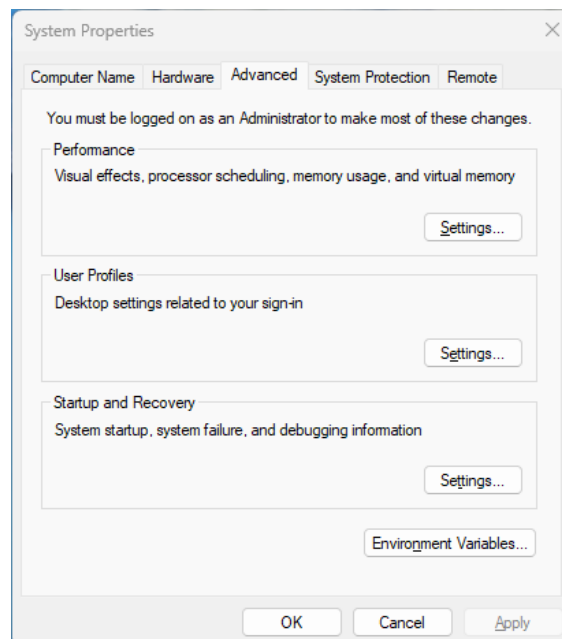
Konfiguracja Licencji:

Po zakończeniu instalacji należy wskazać symulatorowi ścieżkę do licencji. Proces ten różni się w zależności od systemu operacyjnego.

Dla systemu Windows:

Otwórz menu Start i wyszukaj Zaawansowane ustawienia systemu.

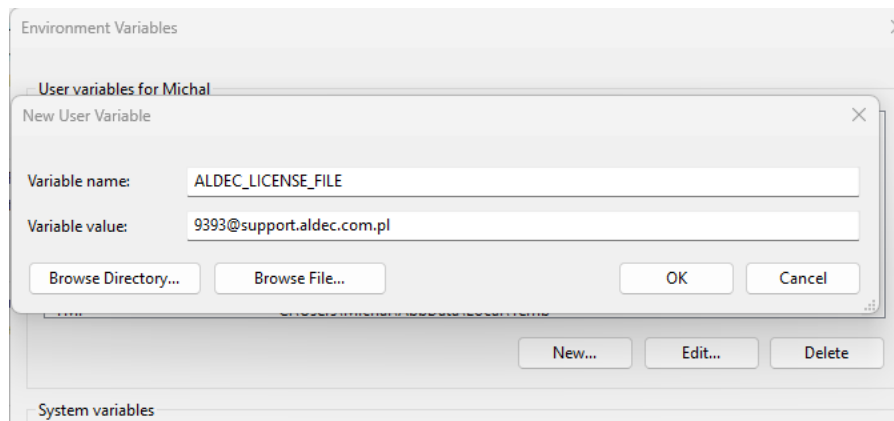
W oknie Właściwości systemu kliknij przycisk Zmienne środowiskowe....



Kliknij Nowa... (najlepiej w sekcji Zmienne użytkownika) i utwórz nową zmienną:

Nazwa zmiennej: ALDEC_LICENSE_FILE

Wartość zmiennej: 9393@support.aldec.com.pl



Zatwierdź wszystkie zmiany przyciskiem OK.

Dla systemu Linux:

Aby ustawić licencję, dodaj odpowiednią zmienną środowiskową do swojego pliku ~/.bashrc. Otwórz plik edytorem tekstowym i dopisz na samym końcu linijkę:

```
export ALDEC_LICENSE_FILE=9393@support.aldec.com.pl
```

Odśwież konfigurację terminala, wykonując komendę:

```
source ~/.bashrc
```

Pierwsze Uruchomienie:

Gdy licencja jest już poprawnie skonfigurowana, środowisko jest gotowe do pracy.

Uruchom symulator:

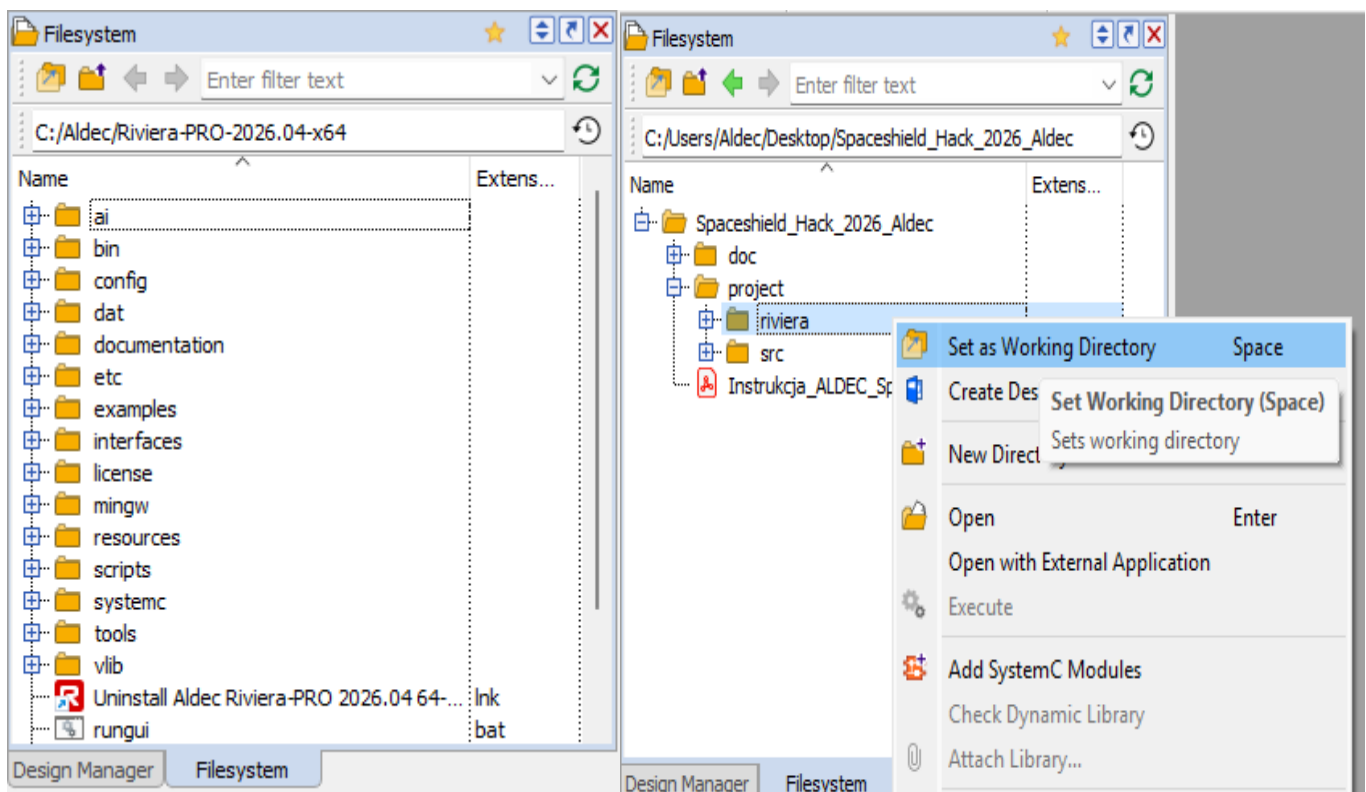
W systemie Windows uruchom aplikację ze skrótu lub z menu Start.

W systemie Linux wpisz w terminalu komendę: riviera.

Ustaw odpowiedni folder roboczy (Working Directory), wybierając ścieżkę:

Spaceshield_Hack_2026_Aldec/project/riviera

Najłatwiej korzystając z zakładki Filesystem:

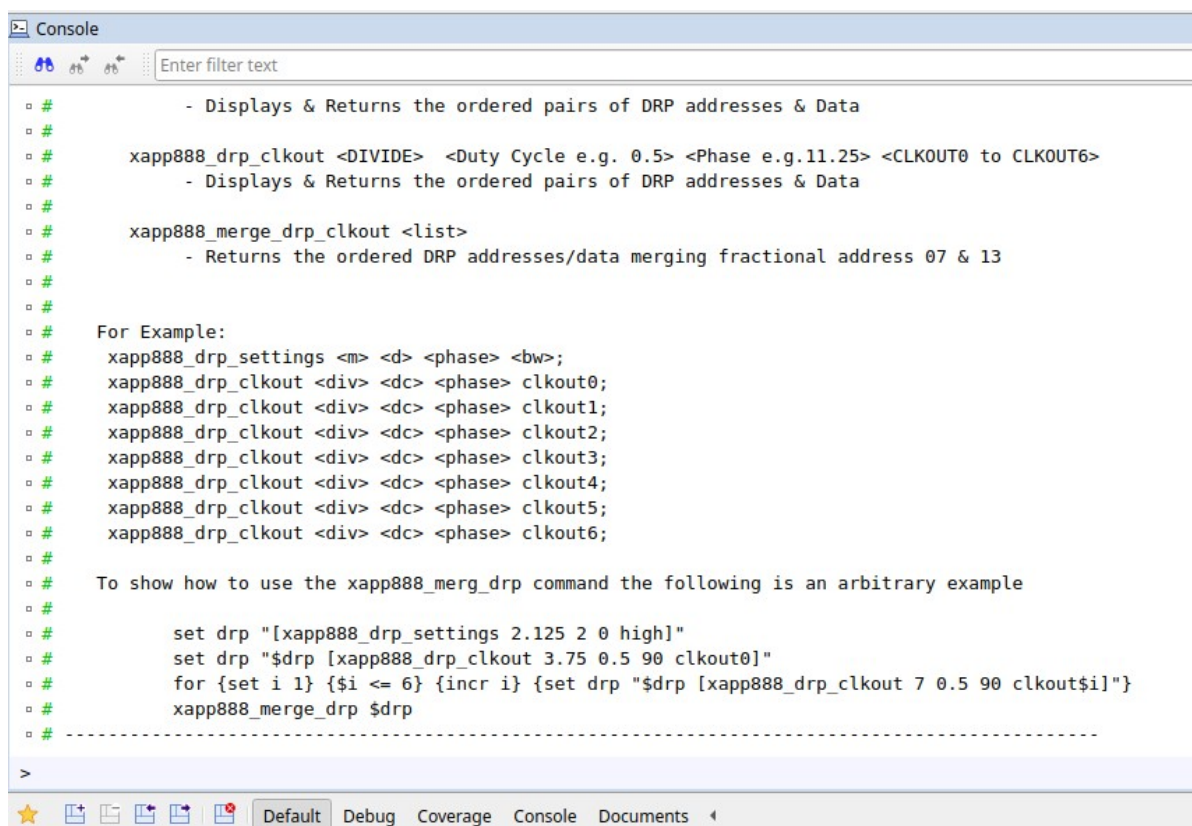


3. Wykonanie skryptów [5 pkt]

W folderze Spaceshield_Hack_2026_Aldec/project/riviera znajdują się 2 pliki: run.do oraz top_mmcme2.tcl. Plik run.do służy do uruchamiania symulacji. Natomiast plik top_mmcme2.tcl jest zbiorem instrukcji, które musimy wykorzystać aby dynamicznie rekonfigurować częstotliwość wyjściową naszego nadajnika. Aby korzystać z instrukcji znajdujących się w pliku top_mmcme2.tcl należy w konsoli symulatora wpisać polecenie:

```
source top_mmcme2.tcl
```

po wykonaniu tego polecenia powinniśmy otrzymać widok, podobny do zamieszczonego na obrazku poniżej:



Nadszedł moment na przejęcie kontroli nad MMCM i ustawienie parametrów przez interfejs DRP.

W konsoli symulatora Riviera-PRO dostępne jest polecenie:

```
xapp888_drp_settings <m> <d> <phase> <bw>
```

Gdzie:

m – mnożnik

d – dzielnik wejściowy

phase – faza sygnału (domyślnie 0)

bw – pasmo (domyślnie OPTIMIZED)

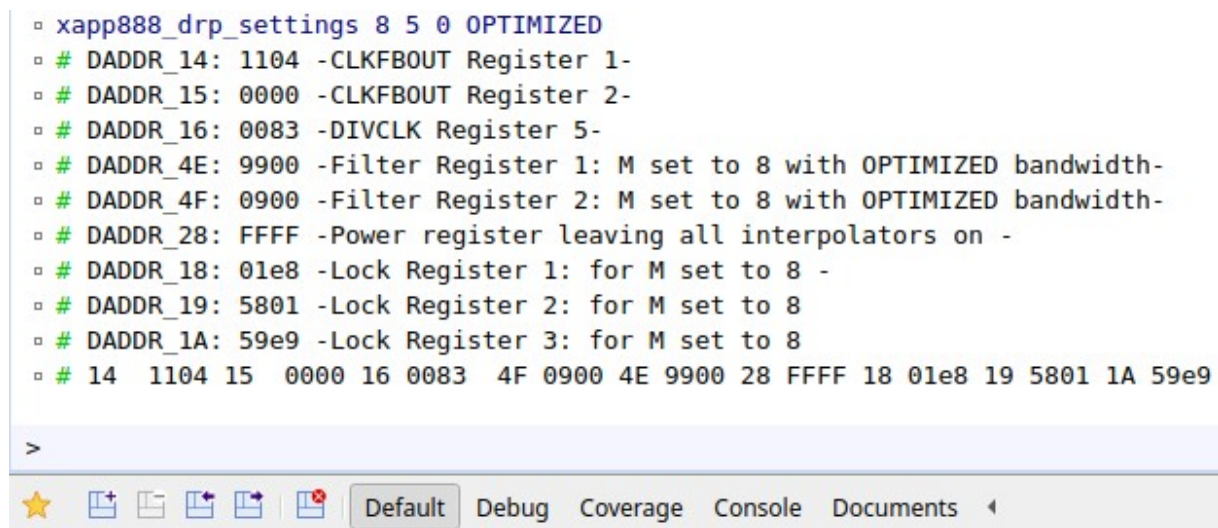
Po uruchomieniu komendy generator wylicza poprawną konfigurację i zwraca listę adresów oraz odpowiadających im danych rejestrów DRP.

Waszym zadaniem będzie zapisanie tych wartości do odpowiednich rejestrów, tak aby MMCM został skonfigurowany zgodnie z wybranymi parametrami i rozpoczął pracę z nowymi ustawieniami.

Na poniższym zrzucie ekranu przedstawiono przykładowy wynik działania polecenia w konsoli Riviera-PRO.

```
xapp888_drp_settings 8 5 0 OPTIMIZED
# DADDR_14: 1104 -CLKFBOUT Register 1-
# DADDR_15: 0000 -CLKFBOUT Register 2-
# DADDR_16: 0083 -DIVCLK Register 5-
# DADDR_4E: 9900 -Filter Register 1: M set to 8 with OPTIMIZED bandwidth-
# DADDR_4F: 0900 -Filter Register 2: M set to 8 with OPTIMIZED bandwidth-
# DADDR_28: FFFF -Power register leaving all interpolators on -
# DADDR_18: 01e8 -Lock Register 1: for M set to 8 -
# DADDR_19: 5801 -Lock Register 2: for M set to 8
# DADDR_1A: 59e9 -Lock Register 3: for M set to 8
# 14 1104 15 0000 16 0083 4F 0900 4E 9900 28 FFFF 18 01e8 19 5801 1A 59e9

>
```



Po ustawieniu częstotliwości wewnętrznej MMCM pozostaje ostatni etap konfiguracji - ustawienie kanału wyjściowego. To właśnie z tego wyjścia zostanie pobrany zegar, którego częstotliwość musi odpowiadać wymaganiom odbiornika statku kosmicznego Artemis III.

W konsoli Riviera-PRO należy użyć polecenia:

```
xapp888_drp_clkout <div> <dc> <phase> clkout0
```

Gdzie:

div – współczynnik podziału

dc – współczynnik wypełnienia (domyślnie 0.5)

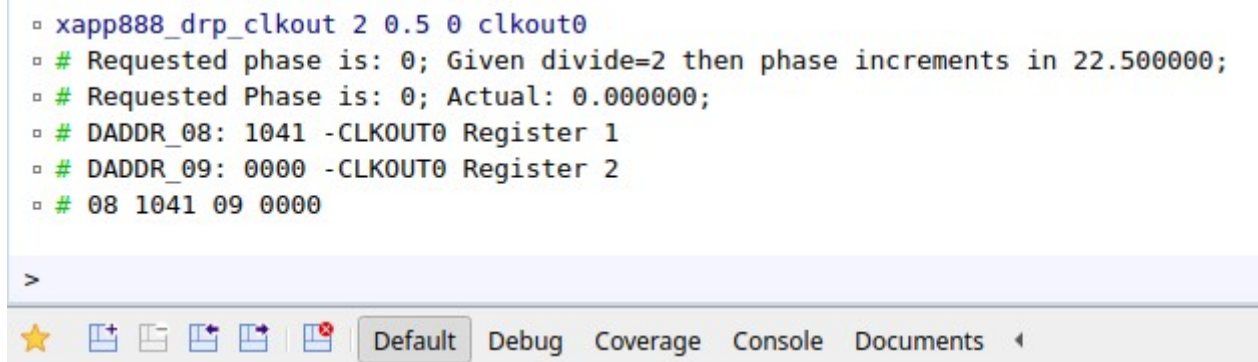
phase – przesunięcie fazowe (domyślnie 0)

Po wykonaniu komendy narzędzie wygeneruje zestaw adresów oraz wartości danych rejestrów DRP, które należy zapisać, aby dokończyć konfigurację MMCM i uzyskać docelowy zegar wyjściowy.

Na poniższym zrzucie ekranu przedstawiono przykładowy wynik działania polecenia w konsoli Riviera-PRO. To właśnie na podstawie takich danych należy przygotować sekwencję zapisów do rejestrów DRP.

```
▫ xapp888_drp_clkout 2 0.5 0 clkout0
▫ # Requested phase is: 0; Given divide=2 then phase increments in 22.500000;
▫ # Requested Phase is: 0; Actual: 0.000000;
▫ # DADDR_08: 1041 -CLKOUT0 Register 1
▫ # DADDR_09: 0000 -CLKOUT0 Register 2
▫ # 08 1041 09 0000

>
```



Do zapisu wyliczonych parametrów do interfejsu DRP należy wykorzystać funkcję drp_write.

Implementację należy wykonać w pliku:

Spaceshield_Hack_2026_Aldec/project/src/TB_mst.v

4. Implementacja modeli BFM oraz parametrów DRP [10pkt]

Kod należy dopisywać wyłącznie w miejscach wyznaczonych do tego celu. Edycja innych fragmentów pliku może spowodować błędy kompilacji, niepoprawne działanie testbenchu lub problemy podczas symulacji.

Z uwagi na awarię radia w misji Artemis III komunikacja działa wyłącznie w jednym kierunku. Oznacza to, że w tym zadaniu wykorzystywany będzie jedynie AXI BFM Master, pełniący rolę stacji nadawczej. Funkcjonalność slave, reprezentująca statek, nie będzie używana.

Aby uzyskać dostęp do interfejsu DRP i móc zapisać wcześniej obliczone wartości, należy najpierw aktywować sygnał resetu modułu MMCM przez magistralę AXI. Po zakończeniu programowania reset należy wyłączyć.

Aby zaprogramować interfejs DRP, należy wykorzystać przygotowaną funkcję drp_write. Funkcja ta przyjmuje dwa parametry: 12-bitowy adres oraz 16-bitową wartość danych, która ma zostać zapisana pod wskazanym adresem. Implementacja drp_write została oparta na mechanizmie BFM firmy Aldec. Wewnątrz funkcji wykorzystano komplet operacji modułu BFM Master dla interfejsu AXI4-Lite, dzięki czemu zapis do DRP realizowany jest w formie pełnej transakcji magistralowej. Aby lepiej zrozumieć sposób działania drp_write, warto przeanalizować fragment kodu zawierający definicję tej funkcji. Znajduje się on w pliku TB_mst.v.

Waszym zadaniem będzie uzupełnienie wskazanych sekcji kodu o kompletną sekwencję operacji:

1. Ustawić reset DRP przy użyciu funkcji AXI BFM Master.
2. Oczekiwać na zakończenie transakcji zapisu.
3. Zapisać wszystkie wyliczone wartości do rejestrów DRP za pomocą funkcji drp_write.
4. Zwolnić reset ponownie przez AXI BFM Master.
5. Oczekiwać na zakończenie transakcji zapisu.
6. Począkać na ponowne zablokowanie MMCM:

```
wait_mmcm_locked();
```

7. Odczekać co najmniej 100 jednostek czasu symulacji.

Parametry resetu DRP:

adres rejestru resetu: 'h40

wartość aktywująca reset: 'h1

wartość dezaktywująca reset: 'h3

Po poprawnym wykonaniu tej sekwencji MMCM powinien rozpocząć pracę z nową konfiguracją.

5. Odczytywanie odpowiedzi od Artemis III [5pkt]

Przed rozpoczęciem testów należy upewnić się, że aktualny katalog roboczy to Spaceshield_Hack_2026_Aldec/project/riviera. Następnie należy uruchomić skrypt run.do, który skompiluje projekt i wystartuje symulację w środowisku Riviera-PRO.

Jeśli wszystkie elementy zostały zaimplementowane poprawnie, w konsoli Riviera-PRO powinna pojawić się informacja o obliczonej częstotliwości wyjściowej. Przykładowy zrzut ekranu znajduje się poniżej.

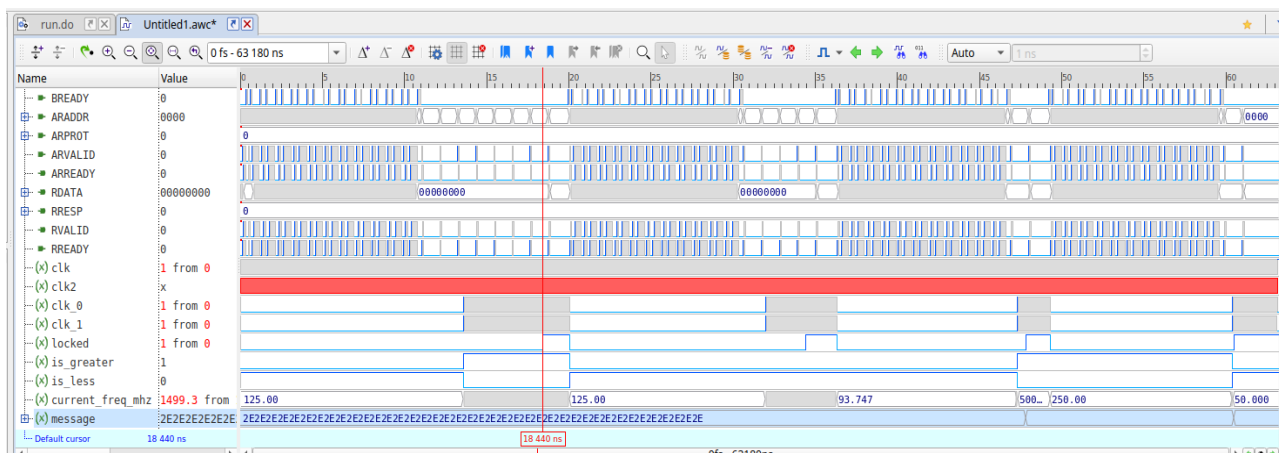
```

❏ run -all
❏ # KERNEL: [11140000] Wait MMCM lock... status=0 (0x00000000)
❏ # KERNEL: Frequency 1499.250375
❏ # KERNEL: MMCM locked after 8 cycles (0x00000001)
❏ # KERNEL: [30720000] Wait MMCM lock... status=0 (0x00000000)
❏ # KERNEL: Frequency 28.846709
❏ # KERNEL: MMCM locked after 5 cycles (0x00000001)
❏ # KERNEL: [47000000] Wait MMCM lock... status=0 (0x00000000)
❏ # KERNEL: Frequency 500.000000
❏ # KERNEL: MMCM locked after 2 cycles (0x00000001)
❏ # KERNEL: [59980000] Wait MMCM lock... status=0 (0x00000000)
❏ # KERNEL: Frequency 50.000000
❏ # KERNEL: MMCM locked after 2 cycles (0x00000001)
❏ # RUNTIME: Info: RUNTIME_0068 TB_mst_test.v (494): $finish called.
❏ # KERNEL: Time: 63180 ns, Iteration: 0, Instance: /TB_mst, Process: @INITIAL#101_0@.
❏ # KERNEL: stopped at time: 63180 ns
❏ # VSIM: Simulation has finished. There are no more test vectors to simulate.

```

Oprócz komunikatów wyświetlanych w konsoli, plik run.do automatycznie generuje plik Untitled1.awc. Po jego otwarciu można zauważyć, że zawiera on przebiegi sygnałów interfejsu AXI4-Lite oraz sygnały pomocnicze wykorzystywane do uzyskania odpowiedzi ze statku Artemis III.

Aby zweryfikować, czy aktualnie ustawiona częstotliwość jest wyższa lub niższa od wartości docelowej, należy przeanalizować przebiegi czasowe wygenerowane przez Riviera-PRO. Przykładowy przebieg przedstawiono poniżej.



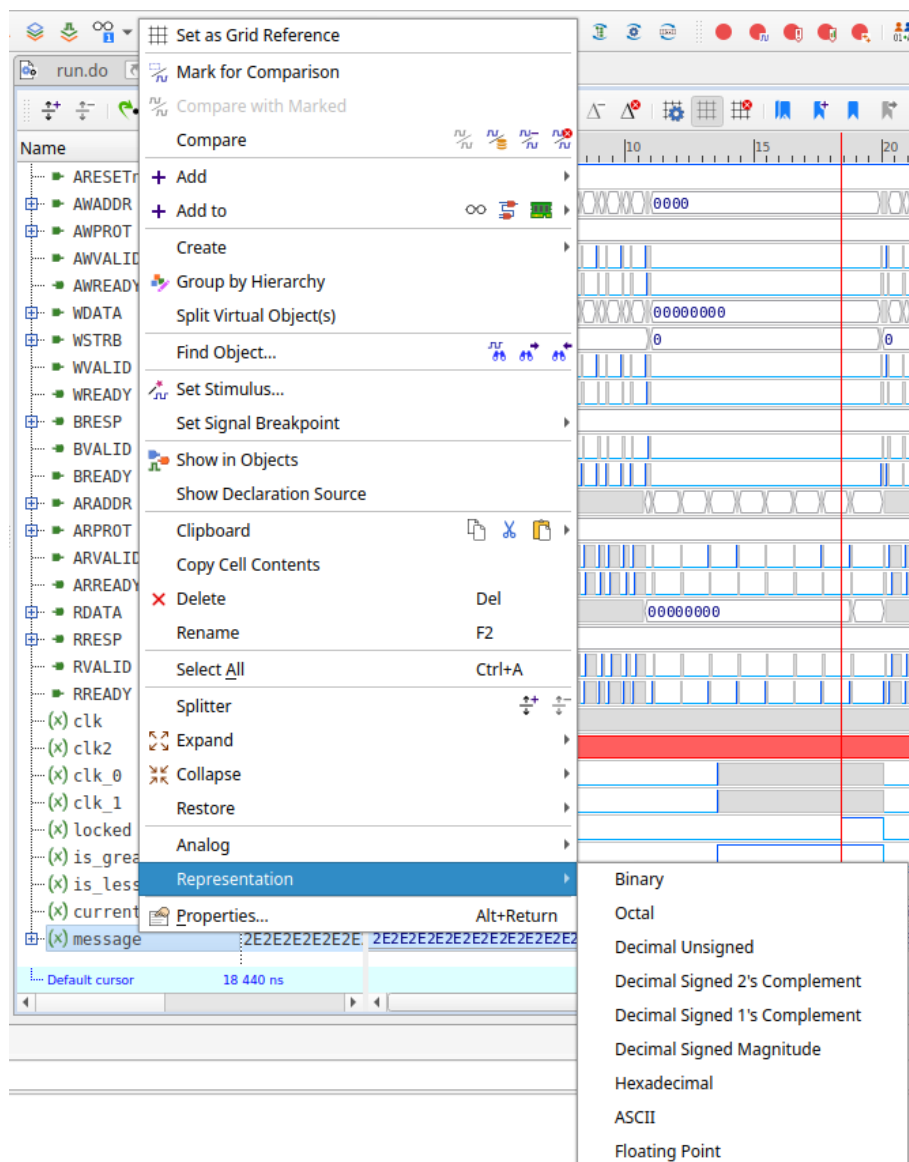
Na przebiegach należy zwrócić szczególną uwagę na trzy sygnały: `is_greater`, `is_less` oraz `message`:

is_less - stan wysoki oznacza, że aktualna częstotliwość jest niższa od wartości docelowej,

is greater - stan wysoki oznacza, że aktualna częstotliwość jest wyższa od wartości docelowej.

Na podstawie tych sygnałów można określić kierunek korekty parametrów MMCM w celu zbliżenia się do właściwej częstotliwości.

Sygnal message reprezentuje odpowiedź ze statku Artemis III. Została ona zakodowana w formacie ASCII, dlatego odebrany komunikat wymaga dekodowania. W symulatorze Riviera-PRO dostępna jest funkcja konwersji danych na przebiegach, którą można wykorzystać do wyświetlenia danych w formacie ASCII.



To już wszystkie informacje, które udało się zdobyć naukowcom z Sol III. Przed Wami ostatni etap zadania. Analizujcie wyniki, korygujcie parametry i nie poddawajcie się po pierwszej nieudanej próbie, inżynieria zawsze opiera się na iteracji. Każde kolejne uruchomienie symulacji przybliży Was do odzyskania łączności z Artemis III.

Zadanie dodatkowe [5 pkt]

Pewien znany naukowiec z planety Sol III został wytypowany do wsparcia ludzkości, aby podobne problemy nie występowały w przyszłości. Wkrótce planowany jest start rakiety, którą naukowiec oraz dwóch innych pasażerów wykorzystają w ramach ważnej misji kosmicznej.

Zadaniem jest opracowanie metody zapewniającej stałą i niezawodną komunikację pomiędzy stacjami kosmicznymi zlokalizowanymi na Sol III a statkami kosmicznymi uczestniczącymi w misjach. Zaproponowane rozwiązanie powinno zostać opisane w sposób jednoznaczny i kompletny, tak aby naukowcy, w razie gdyby zapomnieli, mogli je odtworzyć bez konieczności ponownego opracowywania koncepcji.